DES Pune University, Pune
School of Engineering and Technology
Department of Electronics and Communication Engineering
Program: B. Tech in Electronics and Communication Engineering (AI&ML)

Academic Year: 2026-27 **Year:** Third Year
PRN No.: 1022411079 **Name:** Samruddhi Suryawanshi **Semester:** I
Subject: Database Management System **Assignment No.:** 4
Date: 03/09/2026

Lab Assignment: 04

Title: Advanced SQL Queries: Design and execute following SQL queries on suitable applications.
Joins: Retrieve data from multiple related tables using INNER JOIN, LEFT JOIN, and RIGHT JOIN.
Aggregation: Use aggregate functions like COUNT, AVG, SUM, MIN, MAX in combination with GROUP BY and HAVING clauses.

Theory:

What is Join?

A **JOIN** is an SQL operation used to combine rows from two or more tables based on a related column between them.

For example, the students table and departments table can be joined using the common dept_id column. **Example:**

```
SELECT students.student_name, departments.dept_name
FROM students
INNER JOIN departments
ON students.dept_id = departments.dept_id;
```

This query displays the student name along with the corresponding department name.

Different Types of Join

The main types of JOIN in SQL are:

1. INNER JOIN

Returns only the records that have matching values in both tables.

2. LEFT JOIN

Returns all records from the left table and matching records from the right table. If there is no match, NULL is returned.

3. RIGHT JOIN

Returns all records from the right table and matching records from the left table. If there is no match, NULL is returned.

4. FULL OUTER JOIN

Returns all records from both tables, including matching and non-matching records.

Note: MySQL does not directly support FULL OUTER JOIN; it can be achieved using LEFT JOIN, RIGHT JOIN, and UNION.

5. CROSS JOIN

Returns every possible combination of rows from the two tables.

6. SELF JOIN

A table is joined with itself. It is useful for representing relationships within the same table, such as employees and their managers.

7. NATURAL JOIN

Automatically joins tables using columns having the same name and compatible data types.

8. EQUI JOIN

A join in which the matching condition uses the equality (=) operator.

9. NON-EQUI JOIN

A join in which the condition uses operators other than =, such as >, <, >=, <=, or BETWEEN.

Aggregate Functions

Function	Description
COUNT()	Counts the number of rows
SUM()	Calculates the total
AVG()	Calculates the average
MIN()	Finds the minimum value
MAX()	Finds the maximum value

Examples:

SELECT COUNT(*) FROM students;

Counts the total number of students.

SELECT AVG(marks) FROM students;

Calculates the average marks. SELECT SUM(marks) FROM students;

Calculates the total marks.

SELECT MIN(marks) FROM students; Finds the lowest marks.

SELECT MAX(marks) FROM students; Finds the highest marks.

Group By

The **GROUP BY** clause is used to group rows having the same value in one or more columns.

It is commonly used with aggregate functions such as COUNT(), SUM(), AVG(), MIN(), and MAX().

Example:

SELECT dept_id, COUNT(*) AS total_students

FROM students

GROUP BY dept_id;

This query groups students according to their department and counts the number of students in each department.

Having

The **HAVING** clause is used to filter groups created by the GROUP BY clause.

It is commonly used with aggregate functions.

Example:

SELECT dept_id, COUNT(*) AS total_students

FROM students

GROUP BY dept_id

HAVING COUNT(*) > 1;

This query displays only those departments that have more than one student.

Difference between WHERE and HAVING

- **WHERE** filters individual rows before grouping.
- **HAVING** filters groups after GROUP BY.

Example:

SELECT dept_id, AVG(marks) AS average_marks

FROM students

GROUP BY dept_id

HAVING AVG(marks) > 80;

This displays only departments whose average marks are greater than 80.

Show Query Execution Screenshots for:

Inner join

```
mysql> SELECT students.student_name, departments.dept_name
    -> FROM students
    -> INNER JOIN departments
    -> ON students.dept_id = departments.dept_id;
+--------------+-------------------------+
| student_name | dept_name               |
+--------------+-------------------------+
| Aarav        | Computer Engineering    |
| Rohan        | Computer Engineering    |
| Karan        | Computer Engineering    |
| Ananya       | Electronics Engineering |
| Priya        | Electronics Engineering |
| Sneha        | Mechanical Engineering  |
| Aditya       | Civil Engineering       |
| Vivek        | Information Technology  |
| Neha         | Information Technology  |
+--------------+-------------------------+
9 rows in set (0.00 sec)
```

Left join

```
mysql> SELECT students.student_name, departments.dept_name
    -> FROM students
    -> LEFT JOIN departments
    -> ON students.dept_id = departments.dept_id;
+--------------+------------------------+
| student_name | dept_name              |
+--------------+------------------------+
| Aarav        | Computer Engineering   |
| Ananya       | Electronics Engineering |
| Rohan        | Computer Engineering   |
| Sneha        | Mechanical Engineering |
| Vivek        | Information Technology |
| Priya        | Electronics Engineering |
| Aditya       | Civil Engineering      |
| Neha         | Information Technology |
| Karan        | Computer Engineering   |
| Isha         | NULL                   |
+--------------+------------------------+
10 rows in set (0.00 sec)
```

Right join

```
mysql> SELECT students.student_name, departments.dept_name
    -> FROM students
    -> RIGHT JOIN departments
    -> ON students.dept_id = departments.dept_id;
+--------------+------------------------+
| student_name | dept_name              |
+--------------+------------------------+
| Aarav        | Computer Engineering   |
| Rohan        | Computer Engineering   |
| Karan        | Computer Engineering   |
| Ananya       | Electronics Engineering |
| Priya        | Electronics Engineering |
| Sneha        | Mechanical Engineering |
| Aditya       | Civil Engineering      |
| Vivek        | Information Technology |
| Neha         | Information Technology |
+--------------+------------------------+
9 rows in set (0.00 sec)
```

Cross join

```
mysql> SELECT students.student_name, departments.dept_name
    -> FROM students
    -> CROSS JOIN departments;
+--------------+--------------------------+
| student_name | dept_name                |
+--------------+--------------------------+
| Aarav        | Information Technology   |
| Aarav        | Civil Engineering        |
| Aarav        | Mechanical Engineering   |
| Aarav        | Electronics Engineering  |
| Aarav        | Computer Engineering     |
| Ananya       | Information Technology   |
| Ananya       | Civil Engineering        |
| Ananya       | Mechanical Engineering   |
| Ananya       | Electronics Engineering  |
| Ananya       | Computer Engineering     |
| Rohan        | Information Technology   |
| Rohan        | Civil Engineering        |
| Rohan        | Mechanical Engineering   |
| Rohan        | Electronics Engineering  |
| Rohan        | Computer Engineering     |
| Sneha        | Information Technology   |
| Sneha        | Civil Engineering        |
| Sneha        | Mechanical Engineering   |
| Sneha        | Electronics Engineering  |
| Sneha        | Computer Engineering     |
| Vivek        | Information Technology   |
| Vivek        | Civil Engineering        |
| Vivek        | Mechanical Engineering   |
| Vivek        | Electronics Engineering  |
| Vivek        | Computer Engineering     |
| Priya        | Information Technology   |
| Priya        | Civil Engineering        |
| Priya        | Mechanical Engineering   |
| Priya        | Electronics Engineering  |
| Priya        | Computer Engineering     |
| Aditya       | Information Technology   |
| Aditya       | Civil Engineering        |
```

Self join

```
mysql> SELECT
    ->     e.emp_name AS Employee,
    ->     m.emp_name AS Manager
    -> FROM employees e
    -> LEFT JOIN employees m
    -> ON e.manager_id = m.emp_id;
+----------+---------+
| Employee | Manager |
+----------+---------+
| Rajesh   | NULL    |
| Meena    | Rajesh  |
| Amit     | Meena   |
| Pooja    | Meena   |
| Suresh   | Rajesh  |
| Kavita   | Suresh  |
| Rahul    | Suresh  |
+----------+---------+
7 rows in set (0.00 sec)
```

Natural join

```
mysql> SELECT student_id, student_name, dept_name
    -> FROM students
    -> NATURAL JOIN departments;
+------------+--------------+------------------------+
| student_id | student_name | dept_name              |
+------------+--------------+------------------------+
|        101 | Aarav        | Computer Engineering   |
|        103 | Rohan        | Computer Engineering   |
|        109 | Karan        | Computer Engineering   |
|        102 | Ananya       | Electronics Engineering |
|        106 | Priya        | Electronics Engineering |
|        104 | Sneha        | Mechanical Engineering |
|        107 | Aditya       | Civil Engineering      |
|        105 | Vivek        | Information Technology |
|        108 | Neha         | Information Technology |
+------------+--------------+------------------------+
9 rows in set (0.00 sec)
```

Non-equi join

```
mysql> CREATE TABLE salary_grades (
    ->     grade CHAR(1),
    ->     min_salary DECIMAL(10,2),
    ->     max_salary DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO salary_grades (grade, min_salary, max_salary)
    -> VALUES
    -> ('A', 80000, 100000),
    -> ('B', 60000, 79999),
    -> ('C', 40000, 59999);
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM salary_grades;
+-------+------------+------------+
| grade | min_salary | max_salary |
+-------+------------+------------+
| A     |   80000.00 |  100000.00 |
| B     |   60000.00 |   79999.00 |
| C     |   40000.00 |   59999.00 |
+-------+------------+------------+
3 rows in set (0.00 sec)
```

Full outer join

```
mysql> SELECT
    ->     s.student_name,
    ->     d.dept_name
    -> FROM students s
    -> LEFT JOIN departments d
    -> ON s.dept_id = d.dept_id
    ->
    -> UNION
    ->
    -> SELECT
    ->     s.student_name,
    ->     d.dept_name
    -> FROM students s
    -> RIGHT JOIN departments d
    -> ON s.dept_id = d.dept_id;
+--------------+------------------------+
| student_name | dept_name              |
+--------------+------------------------+
| Aarav        | Computer Engineering   |
| Ananya       | Electronics Engineering |
| Rohan        | Computer Engineering   |
| Sneha        | Mechanical Engineering |
| Vivek        | Information Technology |
| Priya        | Electronics Engineering |
| Aditya       | Civil Engineering      |
| Neha         | Information Technology |
| Karan        | Computer Engineering   |
| Isha         | NULL                   |
+--------------+------------------------+
10 rows in set (0.00 sec)
```

All aggregate functions COUNT() AVG() SUM() MIN() MAX()

DES Pune University, Pune
School of Engineering and Technology
Department of Electronics and Communication Engineering
Program: B. Tech in Electronics and Communication Engineering (AI&ML)

```
mysql> SELECT COUNT(*) AS total_students
    -> FROM students;
+----------------+
| total_students |
+----------------+
|             10 |
+----------------+
1 row in set (0.00 sec)

mysql> SELECT AVG(marks) AS average_marks
    -> FROM students;
+---------------+
| average_marks |
+---------------+
|     82.900000 |
+---------------+
1 row in set (0.00 sec)

mysql> SELECT SUM(marks) AS total_marks
    -> FROM students;
+-------------+
| total_marks |
+-------------+
|      829.00 |
+-------------+
1 row in set (0.00 sec)
```

```
mysql> SELECT MIN(marks) AS minimum_marks
    -> FROM students;
+---------------+
| minimum_marks |
+---------------+
|         69.00 |
+---------------+
1 row in set (0.00 sec)

mysql> SELECT MAX(marks) AS maximum_marks
    -> FROM students;
+---------------+
| maximum_marks |
+---------------+
|         95.00 |
+---------------+
1 row in set (0.00 sec)
```

Group by

```
mysql> SELECT dept_id, COUNT(*) AS total_students
    -> FROM students
    -> GROUP BY dept_id;
+---------+----------------+
| dept_id | total_students |
+---------+----------------+
|    NULL |              1 |
|       1 |              3 |
|       2 |              2 |
|       3 |              1 |
|       4 |              1 |
|       5 |              2 |
+---------+----------------+
6 rows in set (0.00 sec)

mysql> SELECT dept_id, AVG(marks) AS average_marks
    -> FROM students
    -> GROUP BY dept_id;
+---------+---------------+
| dept_id | average_marks |
+---------+---------------+
|    NULL |     89.500000 |
|       1 |     80.000000 |
|       2 |     93.000000 |
|       3 |     88.000000 |
|       4 |     69.000000 |
|       5 |     78.250000 |
+---------+---------------+
6 rows in set (0.00 sec)
```

```
mysql> SELECT dept_id, MAX(marks) AS highest_marks
    -> FROM students
    -> GROUP BY dept_id;
+---------+---------------+
| dept_id | highest_marks |
+---------+---------------+
|    NULL |         89.50 |
|       1 |         85.50 |
|       2 |         95.00 |
|       3 |         88.00 |
|       4 |         69.00 |
|       5 |         82.00 |
+---------+---------------+
6 rows in set (0.00 sec)
```

Having

```
mysql> SELECT
    ->     dept_id,
    ->     COUNT(*) AS total_students
    -> FROM students
    -> GROUP BY dept_id
    -> HAVING COUNT(*) > 1;
+---------+----------------+
| dept_id | total_students |
+---------+----------------+
|       1 |              3 |
|       2 |              2 |
|       5 |              2 |
+---------+----------------+
3 rows in set (0.00 sec)

mysql> SELECT
    ->     dept_id,
    ->     AVG(marks) AS average_marks
    -> FROM students
    -> GROUP BY dept_id
    -> HAVING AVG(marks) > 80;
+---------+---------------+
| dept_id | average_marks |
+---------+---------------+
|    NULL |     89.500000 |
|       2 |     93.000000 |
|       3 |     88.000000 |
+---------+---------------+
3 rows in set (0.00 sec)
```

Set operators UNION INTERSECT DIFFERENCE

```
mysql> SELECT student_name, city
    -> FROM students
    -> WHERE city = 'Pune'
    ->
    -> UNION
    ->
    -> SELECT student_name, city
    -> FROM students
    -> WHERE city = 'Mumbai';
+--------------+--------+
| student_name | city   |
+--------------+--------+
| Aarav        | Pune   |
| Sneha        | Pune   |
| Aditya       | Pune   |
| Karan        | Pune   |
| Ananya       | Mumbai |
| Priya        | Mumbai |
| Isha         | Mumbai |
+--------------+--------+
7 rows in set (0.00 sec)

mysql> SELECT student_id, student_name
    -> FROM students
    -> WHERE city = 'Pune'
    -> AND marks > 80;
+------------+--------------+
| student_id | student_name |
+------------+--------------+
|        101 | Aarav        |
|        104 | Sneha        |
+------------+--------------+
2 rows in set (0.00 sec)
```

```
mysql> SELECT student_id, student_name
    -> FROM students
    -> WHERE city <> 'Pune';
+------------+--------------+
| student_id | student_name |
+------------+--------------+
|        102 | Ananya       |
|        103 | Rohan        |
|        105 | Vivek        |
|        106 | Priya        |
|        108 | Neha         |
|        110 | Isha         |
+------------+--------------+
6 rows in set (0.00 sec)
```

Use of any five date, any five time, any five mathematical and any five string functions

Any five date

```
mysql> SELECT student_name, YEAR(admission_date) AS admission_year
    -> FROM students;
+--------------+----------------+
| student_name | admission_year |
+--------------+----------------+
| Aarav        |           2024 |
| Ananya       |           2023 |
| Rohan        |           2024 |
| Sneha        |           2022 |
| Vivek        |           2023 |
| Priya        |           2024 |
| Aditya       |           2022 |
| Neha         |           2023 |
| Karan        |           2024 |
| Isha         |           2022 |
+--------------+----------------+
10 rows in set (0.00 sec)

mysql> SELECT student_name, MONTH(admission_date) AS admission_month
    -> FROM students;
+--------------+-----------------+
| student_name | admission_month |
+--------------+-----------------+
| Aarav        |               6 |
| Ananya       |               7 |
| Rohan        |               6 |
| Sneha        |               8 |
| Vivek        |               7 |
| Priya        |               6 |
| Aditya       |               8 |
| Neha         |               7 |
| Karan        |               6 |
| Isha         |               8 |
+--------------+-----------------+
10 rows in set (0.00 sec)
```

```
mysql> SELECT student_name, DAY(admission_date) AS admission_day
    -> FROM students;
+--------------+---------------+
| student_name | admission_day |
+--------------+---------------+
| Aarav        |            15 |
| Ananya       |            10 |
| Rohan        |            20 |
| Sneha        |            12 |
| Vivek        |            15 |
| Priya        |            18 |
| Aditya       |            20 |
| Neha         |            25 |
| Karan        |            25 |
| Isha         |            30 |
+--------------+---------------+
10 rows in set (0.00 sec)

mysql> SELECT student_name,
    ->        DATEDIFF(CURDATE(), admission_date) AS days_since_admission
    -> FROM students;
+--------------+----------------------+
| student_name | days_since_admission |
+--------------+----------------------+
| Aarav        |                  808 |
| Ananya       |                 1149 |
| Rohan        |                  803 |
| Sneha        |                 1481 |
| Vivek        |                 1144 |
| Priya        |                  805 |
| Aditya       |                 1473 |
| Neha         |                 1134 |
| Karan        |                  798 |
| Isha         |                 1463 |
+--------------+----------------------+
10 rows in set (0.00 sec)
```

```
mysql> SELECT student_name,
    ->        DATEDIFF(CURDATE(), admission_date) AS days_since_admission
    -> FROM students;
+--------------+----------------------+
| student_name | days_since_admission |
+--------------+----------------------+
| Aarav        |                  808 |
| Ananya       |                 1149 |
| Rohan        |                  803 |
| Sneha        |                 1481 |
| Vivek        |                 1144 |
| Priya        |                  805 |
| Aditya       |                 1473 |
| Neha         |                 1134 |
| Karan        |                  798 |
| Isha         |                 1463 |
+--------------+----------------------+
10 rows in set (0.00 sec)

mysql> SELECT student_name,
    ->        admission_date,
    ->        DATE_ADD(admission_date, INTERVAL 1 YEAR) AS date_after_one_ye
ar
    -> FROM students;
+--------------+----------------+---------------------+
| student_name | admission_date | date_after_one_year |
+--------------+----------------+---------------------+
| Aarav        | 2024-06-15     | 2025-06-15          |
| Ananya       | 2023-07-10     | 2024-07-10          |
| Rohan        | 2024-06-20     | 2025-06-20          |
| Sneha        | 2022-08-12     | 2023-08-12          |
| Vivek        | 2023-07-15     | 2024-07-15          |
| Priya        | 2024-06-18     | 2025-06-18          |
| Aditya       | 2022-08-20     | 2023-08-20          |
| Neha         | 2023-07-25     | 2024-07-25          |
| Karan        | 2024-06-25     | 2025-06-25          |
| Isha         | 2022-08-30     | 2023-08-30          |
+--------------+----------------+---------------------+
10 rows in set (0.00 sec)
```

Any five time

```
mysql> SELECT CURTIME() AS currentTime;
+-------------+
| currentTime |
+-------------+
| 20:37:02    |
+-------------+
1 row in set (0.00 sec)

mysql> SELECT CURRENT_TIME() AS currentTime;
+-------------+
| currentTime |
+-------------+
| 20:37:07    |
+-------------+
1 row in set (0.00 sec)

mysql> SELECT HOUR(CURTIME()) AS currentHour;
+-------------+
| currentHour |
+-------------+
|          20 |
+-------------+
1 row in set (0.00 sec)

mysql> SELECT MINUTE(CURTIME()) AS currentMinute;
+---------------+
| currentMinute |
+---------------+
|            37 |
+---------------+
1 row in set (0.00 sec)

mysql> SELECT SECOND(CURTIME()) AS currentSecond;
+---------------+
| currentSecond |
+---------------+
|            26 |
+---------------+
1 row in set (0.00 sec)
```

Any five Mathematical

```
mysql> SELECT student_name, ROUND(marks) AS rounded_marks
    -> FROM students;
+--------------+---------------+
| student_name | rounded_marks |
+--------------+---------------+
| Aarav        |            86 |
| Ananya       |            91 |
| Rohan        |            79 |
| Sneha        |            88 |
| Vivek        |            75 |
| Priya        |            95 |
| Aditya       |            69 |
| Neha         |            82 |
| Karan        |            76 |
| Isha         |            90 |
+--------------+---------------+
10 rows in set (0.00 sec)

mysql> SELECT student_name, CEIL(marks) AS ceiling_marks
    -> FROM students;
+--------------+---------------+
| student_name | ceiling_marks |
+--------------+---------------+
| Aarav        |            86 |
| Ananya       |            91 |
| Rohan        |            79 |
| Sneha        |            88 |
| Vivek        |            75 |
| Priya        |            95 |
| Aditya       |            69 |
| Neha         |            82 |
| Karan        |            76 |
| Isha         |            90 |
+--------------+---------------+
10 rows in set (0.00 sec)
```

DES Pune University, Pune
School of Engineering and Technology
Department of Electronics and Communication Engineering
Program: B. Tech in Electronics and Communication Engineering (AI&ML)

```
mysql> SELECT student_name, FLOOR(marks) AS floor_marks
    -> FROM students;
+--------------+-------------+
| student_name | floor_marks |
+--------------+-------------+
| Aarav        |          85 |
| Ananya       |          91 |
| Rohan        |          78 |
| Sneha        |          88 |
| Vivek        |          74 |
| Priya        |          95 |
| Aditya       |          69 |
| Neha         |          82 |
| Karan        |          76 |
| Isha         |          89 |
+--------------+-------------+
10 rows in set (0.00 sec)

mysql> SELECT student_name, ABS(marks - 100) AS marks_difference
    -> FROM students;
+--------------+------------------+
| student_name | marks_difference |
+--------------+------------------+
| Aarav        |            14.50 |
| Ananya       |             9.00 |
| Rohan        |            21.50 |
| Sneha        |            12.00 |
| Vivek        |            25.50 |
| Priya        |             5.00 |
| Aditya       |            31.00 |
| Neha         |            18.00 |
| Karan        |            24.00 |
| Isha         |            10.50 |
+--------------+------------------+
10 rows in set (0.00 sec)
```

```
mysql> SELECT student_id, MOD(student_id, 2) AS remainder
    -> FROM students;
+------------+-----------+
| student_id | remainder |
+------------+-----------+
|        110 |         0 |
|        101 |         1 |
|        103 |         1 |
|        109 |         1 |
|        102 |         0 |
|        106 |         0 |
|        104 |         0 |
|        107 |         1 |
|        105 |         1 |
|        108 |         0 |
+------------+-----------+
10 rows in set (0.00 sec)
```

Any five String Functions

```
mysql> SELECT student_name, UPPER(student_name) AS uppercase_name
    -> FROM students;
+--------------+----------------+
| student_name | uppercase_name |
+--------------+----------------+
| Aarav        | AARAV          |
| Ananya       | ANANYA         |
| Rohan        | ROHAN          |
| Sneha        | SNEHA          |
| Vivek        | VIVEK          |
| Priya        | PRIYA          |
| Aditya       | ADITYA         |
| Neha         | NEHA           |
| Karan        | KARAN          |
| Isha         | ISHA           |
+--------------+----------------+
10 rows in set (0.00 sec)

mysql> SELECT student_name, LOWER(student_name) AS lowercase_name
    -> FROM students;
+--------------+----------------+
| student_name | lowercase_name |
+--------------+----------------+
| Aarav        | aarav          |
| Ananya       | ananya         |
| Rohan        | rohan          |
| Sneha        | sneha          |
| Vivek        | vivek          |
| Priya        | priya          |
| Aditya       | aditya         |
| Neha         | neha           |
| Karan        | karan          |
| Isha         | isha           |
+--------------+----------------+
10 rows in set (0.00 sec)
```

```
mysql> SELECT student_name, LENGTH(student_name) AS name_length
    -> FROM students;
+--------------+-------------+
| student_name | name_length |
+--------------+-------------+
| Aarav        |           5 |
| Ananya       |           6 |
| Rohan        |           5 |
| Sneha        |           5 |
| Vivek        |           5 |
| Priya        |           5 |
| Aditya       |           6 |
| Neha         |           4 |
| Karan        |           5 |
| Isha         |           4 |
+--------------+-------------+
10 rows in set (0.00 sec)

mysql> SELECT student_name,
    ->        CONCAT(student_name, ' - ', city) AS student_information
    -> FROM students;
+--------------+---------------------+
| student_name | student_information |
+--------------+---------------------+
| Aarav        | Aarav - Pune        |
| Ananya       | Ananya - Mumbai     |
| Rohan        | Rohan - Nashik      |
| Sneha        | Sneha - Pune        |
| Vivek        | Vivek - Nagpur      |
| Priya        | Priya - Mumbai      |
| Aditya       | Aditya - Pune       |
| Neha         | Neha - Nashik       |
| Karan        | Karan - Pune        |
| Isha         | Isha - Mumbai       |
+--------------+---------------------+
10 rows in set (0.00 sec)
```

```
mysql> SELECT student_name,
    ->        SUBSTRING(student_name, 1, 3) AS first_three_characters
    -> FROM students;
+--------------+------------------------+
| student_name | first_three_characters |
+--------------+------------------------+
| Aarav        | Aar                    |
| Ananya       | Ana                    |
| Rohan        | Roh                    |
| Sneha        | Sne                    |
| Vivek        | Viv                    |
| Priya        | Pri                    |
| Aditya       | Adi                    |
| Neha         | Neh                    |
| Karan        | Kar                    |
| Isha         | Ish                    |
+--------------+------------------------+
10 rows in set (0.00 sec)
```

Conclusion

Thus, we successfully designed and executed various SQL queries using MySQL. We studied and implemented different types of JOINs such as INNER JOIN, LEFT JOIN, RIGHT JOIN, CROSS JOIN, SELF JOIN, FULL OUTER JOIN, NATURAL JOIN, EQUI JOIN, and NON-EQUI JOIN. We also used aggregate functions like COUNT, SUM, AVG, MIN, and MAX along with GROUP BY and HAVING clauses. In addition, we implemented set operators such as UNION, INTERSECT, and DIFFERENCE, and explored Date, Time, Mathematical, and String functions. This assignment helped us understand how SQL can be used to retrieve, combine, group, filter, and manipulate data efficiently.

FAQs:

1. Can aggregate functions be used without the GROUP BY clause?

Yes. Aggregate functions can be used without GROUP BY. In this case, the function operates on all the rows and returns a single result.

Example:

SELECT AVG(marks) AS average_marks

FROM students;

2. What is the difference between COUNT(*) and COUNT(column_name)?

- COUNT(*) counts **all rows**, including rows containing NULL values.
- COUNT(column_name) counts only the rows where the specified column is **NOT NULL**.

Example:

```
SELECT COUNT(*) FROM students;


SELECT COUNT(dept_id) FROM students;
```

3. Can we use multiple aggregate functions in a single query?

Yes. Multiple aggregate functions can be used in the same SQL query.

Example:

```
SELECT
   COUNT(*) AS total_students,
   AVG(marks) AS average_marks,
   MAX(marks) AS highest_marks,
   MIN(marks) AS lowest_marks
FROM students;
```

4. Can we use WHERE and HAVING together in a query with aggregate functions?

Yes. WHERE filters individual rows before grouping, while HAVING filters groups after GROUP BY.

Example:

```
SELECT dept_id, AVG(marks) AS average_marks
FROM students
WHERE marks >= 50
GROUP BY dept_id
HAVING AVG(marks) > 75;
```

Here, WHERE first selects students with marks of 50 or more, and HAVING then selects departments whose average marks are greater than 75.

5. What is the difference between INNER JOIN and LEFT JOIN?

INNER JOIN:

- Returns only rows having matching values in both tables.
- Non-matching rows are excluded.

LEFT JOIN:

- Returns all rows from the left table.
- Matching rows from the right table are included.
- If there is no match, NULL is returned for the right table columns.

6. What is the difference between ON and USING in joins?

- **ON** allows us to specify any join condition explicitly and can be used when the column names are different.
- **USING** is used when both tables have a column with the **same name**.

Using ON:

```
SELECT *
FROM students
JOIN departments
ON students.dept_id = departments.dept_id; Using
```

USING:

```
SELECT *
FROM students
JOIN departments
USING (dept_id);
```

7. Can we use aggregate functions with joins?

Yes. Aggregate functions can be used with joins to calculate results from data stored in multiple related tables.

Example:

```
SELECT d.dept_name,
    COUNT(s.student_id) AS total_students,
    AVG(s.marks) AS average_marks
FROM departments d
LEFT JOIN students s
ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name;
```

This query displays the number of students and their average marks for each department.

DES Pune University, Pune
School of Engineering and Technology
Department of Electronics and Communication Engineering
Program: B. Tech in Electronics and Communication Engineering (AI&ML)